Numerical Method of hlsimII

Asger Limkilde

October 13, 2026

This document describes the numerical method used in the highline simulator. It's is described from an math/numerics point of view. For implementation details not covered here, look in the source code.

Contents

1 Introduction **2**

2 The Numerical Method **2**

2.1 Base model of a highline 2

2.2 Spring constants of webbing 3

2.3 Including a backup 4

2.4 Connecting the highliner 4

2.4.1 Analytical Description of an Object in Free Fall 5

2.4.2 Collision and Event Detection 5

2.5 Dampening 6

2.6 Collisions 6

2.7 Time Integration 6

2.8 Post Processing and Force Computation 6

3 Implementation Details **6**

4 Comparison to the ISA backupfall simulator **6**

5 Conclusion and Future work **6**

A Derivatives to Forces **6**

1 Introduction

My goal with this simulator was to gain a better understanding of the forces involved in highline leash- and backup-falls.

A very useful simulation tool exists already [CITE ISA Backupfall simulator]. However this simulator had some short comings that I wanted to address. Namely, it assumes that the total mass of the system is located at the position of the slackliner. This makes the simulator increasingly inaccurate when lines become longer and heavier. Additionally it would not take into account that in a leash fall, the line flicks up quickly, and some of the energy of the falling slackliner is dissipated through the wave propagation down the line.

So in this work, I was curious to see what would happen if I did a full simulation of the dynamics involved in highline falls. So where [CITE] relied on a simple calculation using energy conservation, in this work I try to do a full time-integration of the highline system when a fall occurs.

This document is structured as follows; In Section 2 I will introduce the numerical model used in this work. In Section 3 I will give a few pointers on how it was implemented, and a few comments on the things that didn't work too well. In Section 4 I will discuss how this method differ from the `ISA backupfall simulator` [CITE] by Augustin Moinat, and look at how the result differ for some examples. Finally in Section 5 I conclude and discuss some future work that could be interesting.

A seperate document will include a validation by comparing in more detail to `ISA backupfall simulator` [CITE] and to real life experiments also by A. Moinat et al.

2 The Numerical Method

2.1 Base model of a highline

The approach we take to model the highline system, is that we imagine that the webbing consists of a chain of springs, with point masses in between. The setup is illustrated in Figure 1. We let $x_i \in \mathbb{R}^2$ denote the position of the ith point mass. For one of these points, there are three forces acting on it; F_i^G, $-F_{i-1}$ and F_i given by the gravity and spring forces calculates as

$$F_i^G = m_i g \tag{1a}$$

$$F_i = k_i e_i \max(||x_i - x_{i+1}|| - l_i, 0) \tag{1b}$$

where k_i is the spring constant of the webbing section, l_i is the base length of the webbing connecting x_i and x_{i+1}. Finally e_i is the unit vector in the direction of the webbing given as

$$e_i = \frac{x_{i+1} - x_i}{||x_{i+1} - x_i||} \tag{2}$$

Finally the maximum function in (1a) makes it so that the webbing acts like a spring when pulled, but without pushing back if pushed together. This mimics a webbing that will stretch when it's pulled but yield without resistance by bending, when pushed.

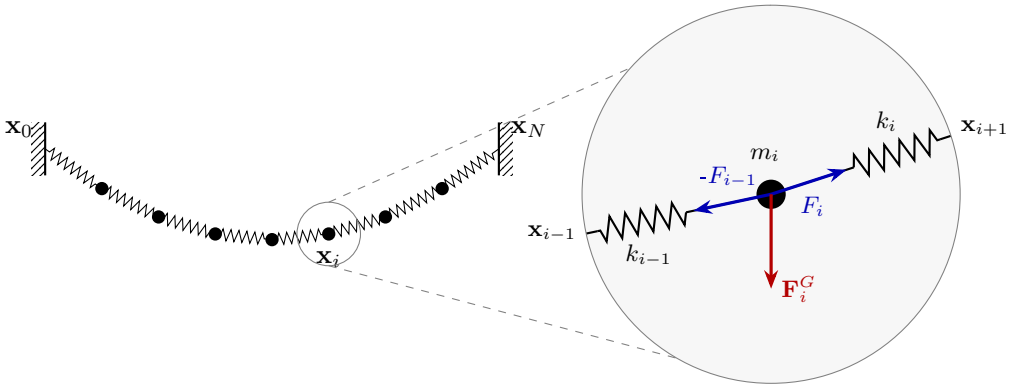


Figure 1: Lumped-mass model of the webbing: point masses connected by springs between the two fixed anchors $\mathbf{x}_0$ and $\mathbf{x}_N$ (left), and the forces acting on an interior mass m_i (right) — the spring forces F_{i-1}, F_i from the two adjacent edges and gravity $\mathbf{F}_G$. Figure by Claude (Anthropic).

Following the zoom-in in the sketch in Figure 1 we can write to net force acting on the point as a sum of these 3 forces.

$$F_i^{\text{net}} = F_i^G - F_{i-1} + F_i \tag{3}$$

Note that each force here has a direction, so $F_i^{\text{net}} \in \mathbb{R}^2$. We now let $x_i(t)$ be the point x_i at time t, to make the notation simpler we just use x_i to denote this, remembering that it is now a function of time. Using Newton's second law $F = ma$ and recalling that acceleration is the double derivative with respect to time $\ddot{x}$ we arrive at the following differential equations:

$$\ddot{x}_i = \frac{F_i^{\text{net}}}{m_i} \tag{4}$$

for $i = 1, \ldots, N$, where N is the number of point masses.

2.2 Spring constants of webbing

To use the model of Section 2.1 we need to estimate the spring constant of a piece of webbing. We will in this work assume a linear stretch curve of the webbing. This is done for multiple reasons; Firstly it is what the ISA backup fall simulator does so it makes a comparison easier. Secondly it make the use of the simulation tool easier. Finally, I don't really trust that the static stress curves that manufactures measure is the same that you would see for a dynamic test, due to internal friction between webbing fibers when it is loaded quickly. Consider a piece of webbing of length l. Then we know that for a percentage of stretch, α, we get the force

$$F_\alpha = k \cdot \alpha \cdot l \tag{5}$$

Slackline companies will often give info of how much their webbing stretches at a given force. For example Joker from slack-inov will stretch 3.6% ($\alpha = 0.036$) at $F_\alpha = 5\text{kN}$. We can use this to compute kl;

$$EA = kl = \frac{F_\alpha}{\alpha} \tag{6}$$

We name this quantity EA as from material science it is the Youngs modulus E times the area of the cross section of the material A. This is (according to Claude) sometimes called the axial stiffness of a rod or rope. So for the Joker example we would get

$$EA_{\text{Joker}} = \frac{5\text{kN}}{0.036} = 138.9\text{kN}$$

Given a value of EA for a specific webbings type we can reconstruct the spring constant for the specific piece of webbing of length l_i by;

$$k_i = \frac{EA}{l_i} \tag{7}$$

2.3 Including a backup

I had some different thoughts we including a backup in the system. In the start I wanted to actually simulate backup loops swinging under the line. However I got a bit discouraged from the extra implementation, and for the fact that I didn't want to consider if the results would also apply for a backup wrapped around the main line (candy-caned). So I decided to model the backup webbing fully algebraic. I model the backup by letting the spring force of a given segment of the line been given by:

$$F_i = e_i\big(k_i^{\text{main}} \max(||\Delta x_i|| - l_i^{\text{main}}, 0) + k_i^{\text{backup}} \max(||\Delta x_i|| - l_i^{\text{backup}}, 0)\big) \tag{8}$$

for $\Delta x_i = x_{i+1} - x_i$. This equation makes it some that if the stretch of the main line is enough to activate the backup line, the two springs both acts on the highline (and we can detect it during simulation). Furthermore, it gives us an easy way to simulate a backup fall; namely we just set the first term in the sum to 0.

2.4 Connecting the highliner

In this section we will describe how the slackliner is added to the numerical model. The basic idea can be summarised the following:

(A) We assume that the leash is rigid and attatched to a specific node of the model throughout the simulation.

(B) We ignore any movement of the slackliner in the x-direction. So we simply model his height during the fall.

(C) While in free fall we compute his height analytically.

(D) While attatched to the line we simply add his mass to the node of attatchment

Some comments on these modelling choices;

(A) Using a rigid leash avoids us tracking the stretch of the leash explicitely. A leash is often short (relative to the mesh of the highline) and much stiffer, as it combines rope and webbing. Firstly, we don't have good information on how they stretch, secondly it make the differential equation much slower to solve as it adds a very stiff element [Refer to some book on stiff solvers]. Additionally I found during experimentation, that we anyways need to handle the collision point where the leash gets taught, to avoid a spurious spike in leash force at this moment.

(B) The reason to ignore the movement of the slackliner in the x-direction, is that I don't have a clean and efficient method of modelling an arbitrary connection point. And once we fix the node of attatchement, modelling lateral movement too will introduce swinging of the slackliner during a backup fall where the line shifts. This is not what

happens in real life, as the leash ring slides with some friction. But as this would be very hard to model accurately, I chose to only model the height of the slackliner - which is what we are the most interested in anyways.

(C) Computing his state analytically (contrasted to adding his state to the differential equation), keeps a super clean structure of equations (every node is attatched to it's two neighbors), which can be used to speed up the numerics of the solver significantly. See [Refer to structured Jacobian block sparse Appendix].

(D) Same argument as (C).

2.4.1 Analytical Description of an Object in Free Fall

A free falling object with starting position y_0 and velocity v_0 at time t_0 can be computed as

$$y(t) = -\frac{1}{2}mg(t - t_0)^2 + v_0(t - t_0) + y_0 \tag{9a}$$

$$v(t) = -mg(t - t_0) + v_0 \tag{9b}$$

2.4.2 Collision and Event Detection

To make this model work we need to do event detection during the time integration of (4). Firstly when the leash gets taught, ie. when the slackliner is l_{leash} under the node of attatchement, we need to detect it. To resolve the collision we modify the velocity of the node, in a way that preserves momentum:

$$v_{i_s} = \frac{v_{i_s} m_n + v_s m_s}{m_n + m_s} \tag{10}$$

where i_s is the node of attatchement, m_n is the mass of said node, v_s is the velocity of the slackliner and m_s is their mass.

Note that this will lead to an energy loss of the system, as we have used an inexact collision to resolve the event. This could lead to an under estimation of the severity of the fall, but the energy loss decreases when the number of nodes increases (for more fine resultions) and it also reflects what happens in real life; some energy is lost to heat or propegated as waves through the line with the collision of the leash ring on the highline.

This event can easily be monitored during the time integration by checking when $y(t) = y_{i_s} - l_{\text{leash}}$. Once detected we change the velocity of the node of attatchement using (10) and add the mass of the slackliner to the node.

We do have to detect when the leash might go loose again (the slackliner goes back to free fall). This is simply done by checking whether the downward acceleration of the node of attatchement exceeds the gravitational acceleration. At this event we go back to the analytical description of the slackliners height, and sets the mass of the attatchement node back to it's node mass.

2.5 Dampening

2.6 Collisions

2.7 Time Integration

2.8 Post Processing and Force Computation

3 Implementation Details

4 Comparison to the ISA backupfall simulator

5 Conclusion and Future work

A Derivatives to Forces

B ODE Solver